

PROGRAM - 3

K-NN Classification Using Public Domain Dataset

AIM

Program to implement k-NN classification using any standard dataset available in the public domain and find the accuracy of the algorithm.

ALGORITHM

1. Import KNeighborsClassifier and load_iris from sklearn.
2. Load the Iris dataset.
3. Assign X = irisdata.data and Y = irisdata.target.
4. Split the dataset into training and testing sets.
5. Create a KNN model with one neighbor using KNeighborsClassifier(n_neighbors=1).
6. Train the model using fit(X_train, Y_train).
7. Predict the class labels for the test data using predict(X_test).
8. Predict the class for a new data sample and display the result.

PROGRAM

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
irisdata=load_iris()
print(irisdata.data)
x=irisdata.data
y=irisdata.target
x_train,x_test,y_train,y_test=train_test_split(x,y)
knn=KNeighborsClassifier(n_neighbors=1)
knn.fit(x_train,y_train)
print(x_test)
print(knn.predict(x_test))
print(y_test)
result=knn.predict([[2,4,6,2]])
print(irisdata.target_names[result])
```

OUTPUT

```
[[5.1 3.5 1.4 0.2]
 [4.9 3. 1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5. 3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [5.1 2.5 3. 1.1]
 [5.8 2.7 5.1 1.9]
 [6.2 3.4 5.4 2.3]
 [6.1 3. 4.9 1.8]
 [7.4 2.8 6.1 1.9]
 [5.5 2.5 4. 1.3]]
[0 2 0 1 0 1 2 2 2 0 0 2 1 0 2 2 2 2 1 1 0 2 2 2 2 2 0 1 0 2 2 1 1 1 0 0 0
 1]
[0 2 0 1 0 1 2 2 2 0 0 2 1 0 2 2 2 2 2 1 0 2 2 2 2 2 0 1 0 2 2 2 1 1 0 0 0
 1]
['virginica']
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 5

Naive Bayes Classification using Public Domain Dataset

AIM

Program to implement Naive Bayes classification using any dataset Input:

ALGORITHM

1. Start
2. Import the required libraries such as `sklearn.datasets`, `sklearn.model_selection`, and `sklearn.naive_bayes`.
3. Load the Iris dataset using `load_iris()` and store features in `X` and labels in `y`.
4. Split the dataset into training and testing sets using `train_test_split()`.
5. Create a Gaussian Naive Bayes (GNB) model using `GaussianNB()`.
6. Train the model using the training data with `.fit()`.
7. Predict the class labels for the test data using `.predict()`.
8. Display the predicted results.
9. Give a new input sample and predict its class label using the trained model.
10. Find the accuracy using the `.score()` function.
11. Display the accuracy of the model.
12. Stop

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
X,y=load_iris(return_X_y=True)
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.5,random_state=0)
gnb=GaussianNB()
y_pred=gnb.fit(X_train,y_train).predict(X_test)
print(y_pred)
x_new=[[5,5,4,4]]
y_new=gnb.fit(X_train,y_train).predict(x_new)
print("predicted output for [[5,5,4,4]]:",y_new)
```

```
print("Naive Bayes score:",gnb.score(X_test,y_test))
```

OUTPUT

```
[2 1 0 2 0 2 0 1 1 1 1 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0
 1 1 1 2 0 2 0 0 1 2 2 1 2 1 2 1 1 2 1 2 1 0 2 1 1 1 1 2 0 0 2 1 0 0
 1]
predicted output for [[5,5,4,4]]: [2]
Naive Bayes score: 0.9466666666666667
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 7

Linear Regression using Public Domain Dataset

AIM

To implement Linear Regression on a public domain dataset and predict the output for a new input value.

ALGORITHM

1. Import the required libraries numpy and LinearRegression from sklearn.linear_model.
2. Create the input data array X and output data array y.
3. Create a Linear Regression model using LinearRegression().
4. Train the model using the fit(X, y) function.
5. Display the coefficient of the model using coef_.
6. Display the intercept of the model using intercept_.
7. Predict the output for a new input value using the predict() function.
8. Display the predicted result

PROGRAM

```
from sklearn.linear_model import LinearRegression
import numpy as np
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 5, 4, 5])
#Train model
model = LinearRegression().fit(X, y)
#Predict and show results
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Prediction for x=6:", model.predict([[6]])[0])
```

OUTPUT

```
PS C:\Users\MCETCSE\Desktop\dsm12> python linear2.py  
Coefficient: 0.6  
Intercept: 2.2  
Prediction for x=6: 5.8  
PS C:\Users\MCETCSE\Desktop\dsm12>
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 9

Multiple Regression using Public Domain Dataset

AIM

To implement Multiple Linear Regression using selected features from the diabetes dataset to predict disease progression and evaluate the model performance.

ALGORITHM

1. Start.
2. Import required libraries:
 - scikit-learn for regression
 - pandas for data handling
 - matplotlib for visualization
3. Load the diabetes dataset and convert it into a pandas DataFrame.
4. Select independent variables (features) and the dependent variable (target).
5. Split the dataset into training and testing sets using `train_test_split()`.
6. Create a Linear Regression model and train it with the training data using `.fit()`.
7. Print the regression equation (intercept + coefficients).
8. Predict the target values for the test data.
9. Evaluate the model using Mean Squared Error (MSE) and R^2 score.
10. Predict the target value for new input data.
11. Visualize actual vs predicted values using a scatter plot.
12. Stop.

PROGRAM

```
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.datasets import load_diabetes

from sklearn.metrics import mean_squared_error, r2_score

import pandas as pd

import matplotlib.pyplot as plt

diabetes = load_diabetes(as_frame=True)
```

```

df = diabetes.frame

print("Dataset Loaded Successfully!\n")

print(df.head())

X = df[['bmi', 'bp', 's1', 's5']]

y = df['target']

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = LinearRegression()

model.fit(x_train, y_train)

print("\n Regression Equation:")

print(f'y = {model.intercept_:.2f}', end='')

for feature, coef in zip(X.columns, model.coef_):

    print(f' + ({coef:.2f} × {feature})', end='')

print("\n")

y_pred = model.predict(x_test)

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print(" Model Performance Metrics:")

print("Mean Squared Error (MSE):", round(mse, 2))

print("R2 Score:", round(r2, 2))

new_data = pd.DataFrame([[0.05, 0.03, 0.02, 0.04]], columns=['bmi', 'bp', 's1', 's5'])

predicted_value = model.predict(new_data)

print("\n Predicted Target Value for New Data:", round(predicted_value[0], 2))

plt.figure(figsize=(6,5))

plt.scatter(y_test, y_pred, color='blue', s=60)

plt.xlabel("Actual Values")

plt.ylabel("Predicted Values")

plt.title("Actual vs Predicted (Multiple Linear Regression)")

```

```
plt.grid(True)
```

```
plt.show()
```

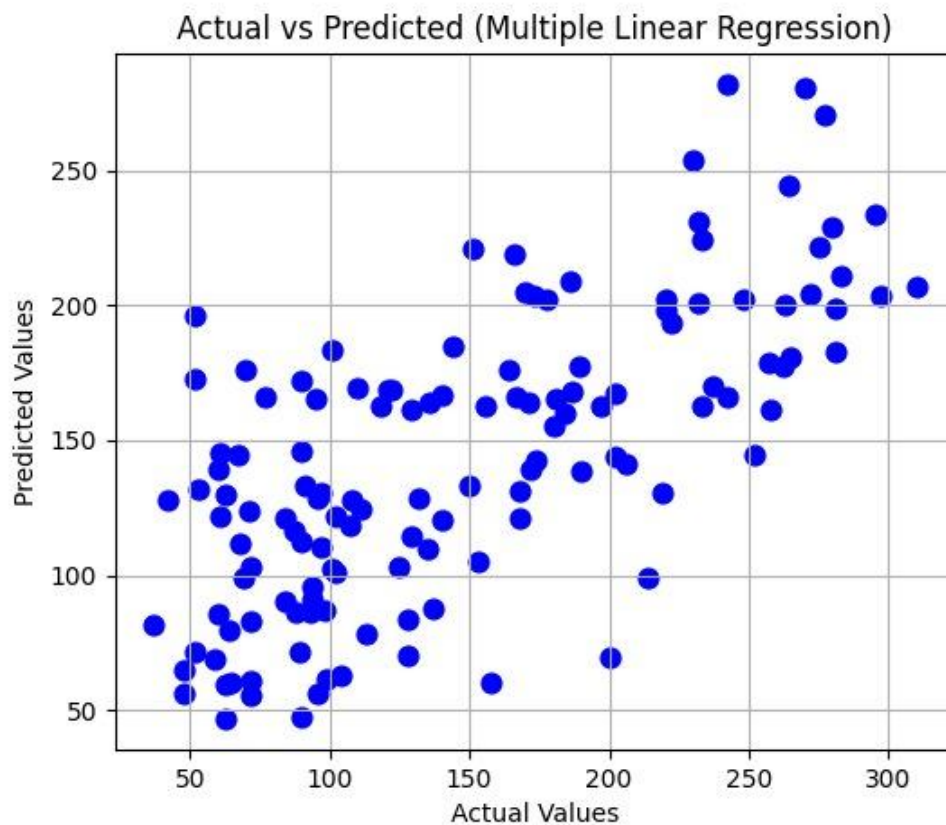
OUTPUT

```
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356 -0.002592  0.002861 -0.025930  141.0
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038  0.034309  0.022688 -0.009362  206.0
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142 -0.002592 -0.031988 -0.046641  135.0
```

```
Regression Equation:
y = 150.84 + (669.56 x bmi) + (326.92 x bp) + (-235.49 x s1) + (561.27 x s5)
```

```
Model Performance Metrics:
Mean Squared Error (MSE): 2799.73
R2 Score: 0.48
```

```
Predicted Target Value for New Data: 211.87
```



RESULT: Program executed successfully and output is verified.

PROGRAM -11

Support Vector Machine

AIM

To implement Support Vector Machine (SVM) classification using the Breast Cancer dataset.

ALGORITHM

1. Import required libraries from sklearn: datasets, train_test_split, metrics, and svm.
2. Load the breast cancer dataset using datasets.load_breast_cancer().
3. Split the dataset into training and testing sets using train_test_split() with 30% test size.
4. Create a Support Vector Machine classifier with a linear kernel using svm.SVC(kernel='linear').
5. Train the classifier using the training data with fit().
6. Predict the class labels for the test data using predict().
7. Print the actual and predicted class labels.
8. Calculate and print performance metrics-accuracy, precision and recall

PROGRAM

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn import svm
cancer = datasets.load_breast_cancer()
x_train, x_test, y_train, y_test = train_test_split( cancer.data,
cancer.target, test_size=0.3, random_state=109)
clf = svm.SVC(kernel='linear')
clf.fit(x_train, y_train)
y_pred = clf.predict(x_test)
print("Actual values:", y_test)
print("Predicted values:", y_pred)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
print("Precision:", metrics.precision_score(y_test, y_pred))
print("Recall:", metrics.recall_score(y_test, y_pred))
```

OUTPUT

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell
PS C:\DSML ANJU> python svm.py
Actual values: [1 1 0 0 1 0 1 1 1 0 0 0 1 0 1 1 0 0 1 0 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1
0 1 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 1 0 1 1 1 0 0 1 0 1 0 0 1 1 1 1 0 1 1 1
0 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 1 0 1 1
0 1 1 0 0 0 1 0 1 1 1 1 1 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 1 0 1 1 0 0 1 0
0 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 1 1 0 1 1 1]
Predicted values: [1 1 0 0 1 0 1 1 1 0 0 0 1 0 0 1 0 0 1 0 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1
0 1 1 0 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 1 1 1 0 0 1 0 1 0 0 1 1 1 1 0 1 1 1
0 1 0 1 1 1 1 1 1 1 0 0 1 1 1 1 0 0 0 0 1 1 1 1 1 0 0 1 0 0 1 1 0 1 0 1 1
0 1 1 0 0 0 1 0 1 1 1 1 1 0 1 0 1 0 1 0 0 0 1 0 1 1 0 1 1 1 0 1 1 0 0 1 0
0 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 1 1 0 1 1 1]
Accuracy: 0.9649122807017544
Precision: 0.9811320754716981
Recall: 0.9629629629629629
PS C:\DSML ANJU> 
```

RESULT: Program executed successfully and output is verified.

PROGRAM-12

Decision Trees using Public Domain Dataset

AIM

To implement a Decision Tree Classifier using the Iris dataset (public domain) and visualize the trained decision tree.

ALGORITHM

1. Import necessary libraries: load_iris from sklearn.datasets, DecisionTreeClassifier from sklearn.tree, and matplotlib.pyplot.
2. Load the Iris dataset using load_iris().
3. Assign the feature data to X and target labels to y.
4. Create a Decision Tree model using DecisionTreeClassifier().
5. Train the model using fit(X, y).
6. Plot the trained decision tree using tree.plot_tree() with filled colors, feature names, and class names.
7. Display the plotted decision tree using plt.show().

PROGRAM

```
from sklearn.datasets import load_iris

from sklearn import tree

import matplotlib.pyplot as plt

iris = load_iris()

X, y = iris.data, iris.target

clf = tree.DecisionTreeClassifier()

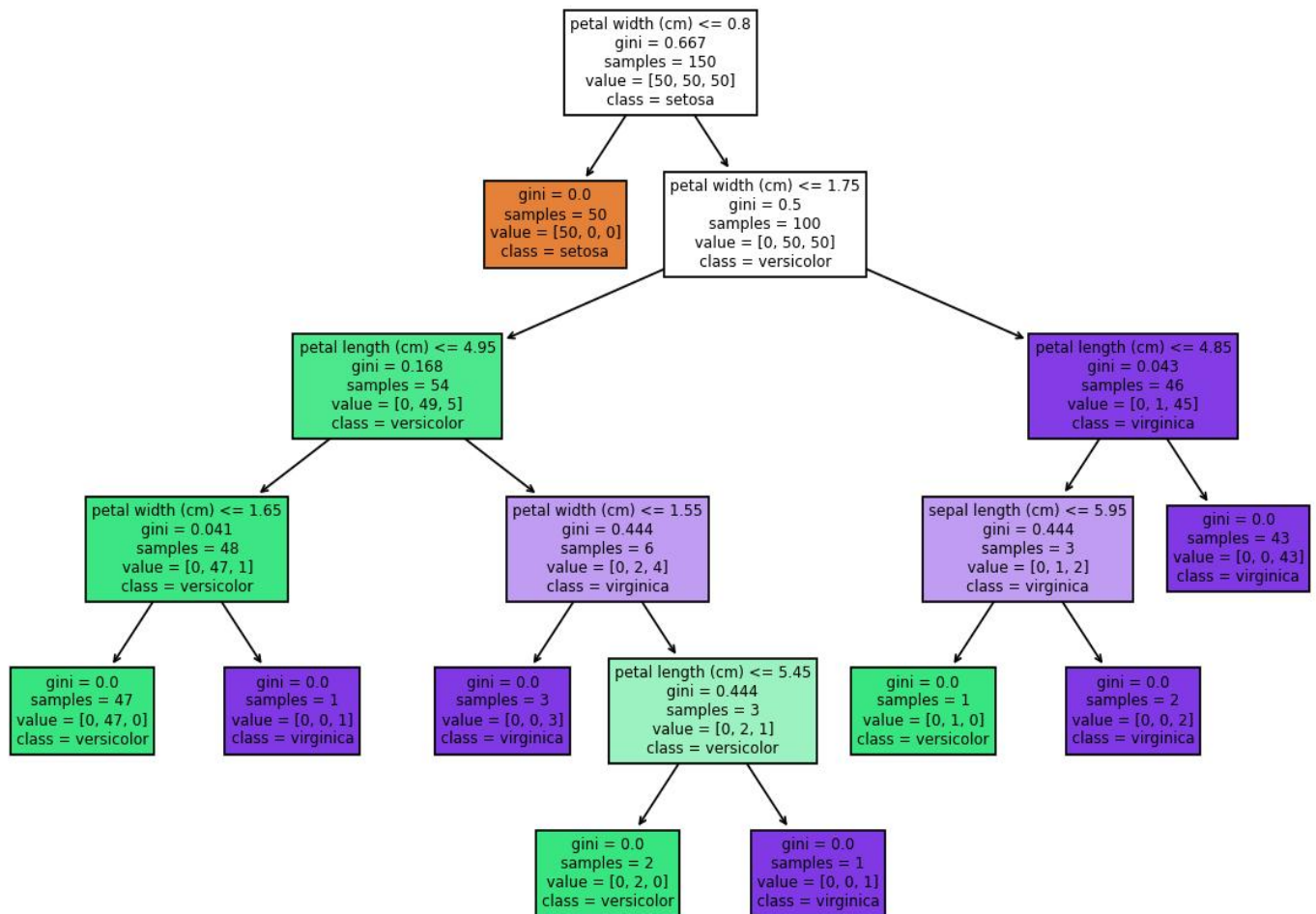
clf = clf.fit(X, y)

plt.figure(figsize=(12,8))

tree.plot_tree(clf, filled=True, feature_names=iris.feature_names, class_names=iris.target_names)

plt.show()
```

OUTPUT



RESULT: Program executed successfully and output is verified.

PROGRAM -14

K-Means Clustering using Public Domain Dataset

AIM

To implement K-Means Clustering on the Iris dataset (public domain) to group the data points into clusters and visualize the cluster distribution along with centroids.

ALGORITHM

1. Import necessary libraries like numpy, pandas, matplotlib, KMeans.
2. Load the Iris dataset.
3. Select the features (Petal Length and Petal Width) as X.
4. Create a K-Means model with 3 clusters.
5. Fit the model and predict cluster labels.
6. Plot the data points colored by cluster.
7. Plot the cluster centroids.
8. Add titles, labels, legend, and display the plot.

PROGRAM

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

from sklearn.datasets import load_iris

iris = load_iris()

X = iris.data[:, [2, 3]]

df = pd.DataFrame(X, columns=['Petal Length', 'Petal Width'])

kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)

y_kmeans = kmeans.fit_predict(X)

plt.figure(figsize=(8, 6))
```

```
plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s=100, c='red', label='Cluster 1')
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s=100, c='blue', label='Cluster 2')
plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s=100, c='green', label='Cluster 3')
plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1], s=300, c='yellow',
marker='*', label='Centroids')

plt.title('K-Means Clustering on Iris Dataset')

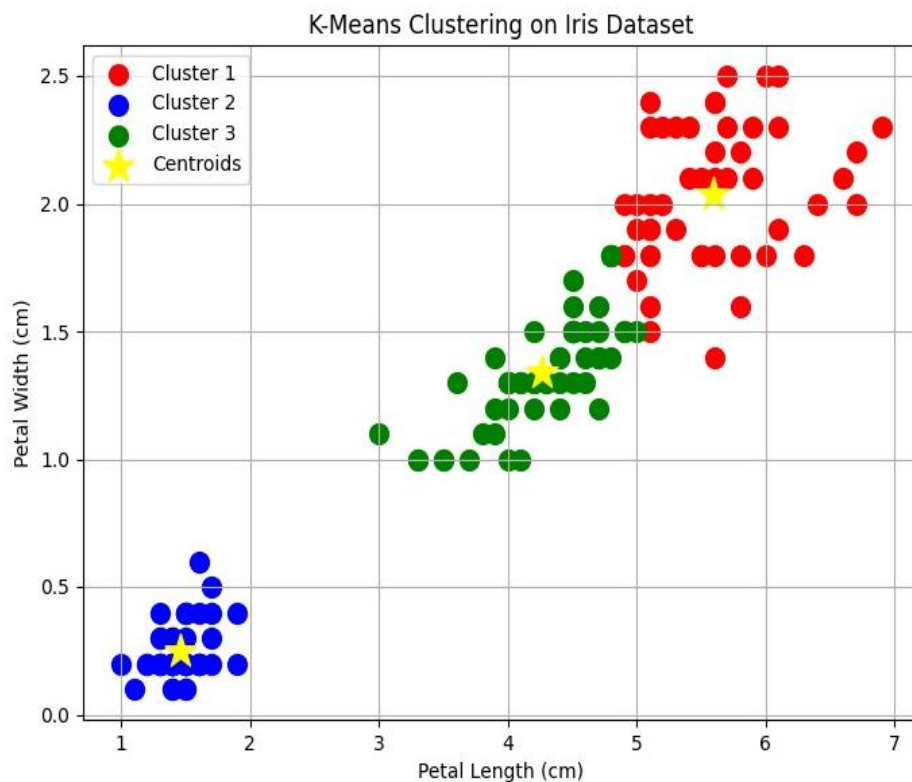
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')

plt.legend()

plt.grid(True)

plt.show()
```

OUTPUT



RESULT: Program executed successfully and output is verified.

PROGRAM 16

Convolutional Neural Network using Public Domain Dataset

AIM

To build and train a Convolutional Neural Network (CNN) using the Keras framework to classify images from the MNIST dataset.

ALGORITHM

1. Start
2. Import libraries like TensorFlow and Keras modules (datasets, layers, models).
3. Load the dataset: MNIST handwritten digits dataset.
4. Preprocess the data:
 - Reshape images to include channel dimension (28, 28, 1).
 - Normalize pixel values to the range [0,1].
5. Build the CNN model:
 - Add a Conv2D layer with ReLU activation.
 - Add a MaxPooling2D layer.
 - Add another Conv2D + MaxPooling2D layer.
 - Flatten the output.
 - Add a Dense layer with 64 neurons (ReLU).
 - Add output Dense layer with 10 neurons (softmax) for 10 classes.
6. Compile the model: Use Adam optimizer, sparse categorical cross-entropy loss, and accuracy metric.
7. Train the model on training data and validate using test data.
8. Evaluate the model on test data.
9. Print test accuracy.
10. Stop

PROGRAM

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models

(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

x_train = x_train.reshape((x_train.shape[0], 28, 28, 1))
```

```

x_test = x_test.reshape((x_test.shape[0], 28, 28, 1))

x_train, x_test = x_train / 255.0, x_test / 255.0

model = models.Sequential([

layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),

layers.MaxPooling2D((2, 2)),

layers.Conv2D(64, (3, 3), activation='relu'),

layers.MaxPooling2D((2, 2)),

layers.Flatten(),

layers.Dense(64, activation='relu'),

layers.Dense(10, activation='softmax') # 10 classes (digits 0–9)])

model.compile(optimizer='adam',

loss='sparse_categorical_crossentropy',

metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)

print(f"Test Accuracy: {test_acc:.4f}")

```

OUTPUT

```

Epoch 1/5
1875/1875 ————— 34s 17ms/step - accuracy: 0.9563 - loss: 0.1431 - val_accuracy: 0.9831 - val_loss: 0.0507
Epoch 2/5
1875/1875 ————— 31s 17ms/step - accuracy: 0.9852 - loss: 0.0468 - val_accuracy: 0.9890 - val_loss: 0.0331
Epoch 3/5
1875/1875 ————— 42s 17ms/step - accuracy: 0.9898 - loss: 0.0329 - val_accuracy: 0.9899 - val_loss: 0.0286
Epoch 4/5
1875/1875 ————— 32s 17ms/step - accuracy: 0.9924 - loss: 0.0236 - val_accuracy: 0.9892 - val_loss: 0.0302
Epoch 5/5
1875/1875 ————— 31s 17ms/step - accuracy: 0.9944 - loss: 0.0173 - val_accuracy: 0.9904 - val_loss: 0.0311
313/313 - 2s - 8ms/step - accuracy: 0.9904 - loss: 0.0311
Test Accuracy: 0.9904

```

RESULT: Program executed successfully and output is verified.

PROGRAM - 19

NLP Implementation using NLTK

AIM

To implement Natural Language Processing tasks such as Part-of-Speech tagging, N-gram generation, smoothing, and chunking using NLTK

ALGORITHM

1. Start.
2. Import NLTK libraries for tokenization, POS tagging, N-grams, stop words, stemming, smoothing, and chunking.
3. Download required NLTK resources (punkt, averaged_perceptron_tagger, stopwords).
4. Input the text to process.
5. Tokenize the text into words.
6. Remove stop words from the tokens.
7. Stem the filtered words to their root forms.
8. Perform Part-of-Speech (POS) tagging on the tokens.
9. Count occurrences of each POS tag.
10. Generate bigrams and trigrams from the tokens.
11. Calculate bigram frequencies.
12. Apply Lidstone smoothing to estimate probabilities of bigrams.
13. Define a grammar and chunk the tagged sentence to extract phrases (e.g., noun phrases).
14. Display all results (tokens, filtered words, stemmed words, POS tags, N-grams, frequencies, smoothed probabilities, chunked tree).
15. Stop.

PROGRAM

```
import nltk

from nltk import word_tokenize, pos_tag, bigrams, trigrams

from nltk.corpus import stopwords

from nltk.stem import PorterStemmer

from nltk.probability import FreqDist, LidstoneProbDist
```

```

from nltk import RegexpParser

from collections import Counter

nltk.data.path.append("C:/Desktop/nltk_data")

nltk.download('punkt', download_dir="C:/Desktop/nltk_data")

nltk.download('averaged_perceptron_tagger', download_dir="C:/Desktop/nltk_data")

nltk.download('stopwords', download_dir="C:/Desktop/nltk_data")

text = "Tomatoes are essential in cooking. They grow well in warm climates."

print("Original Text:\n", text)

tokens = word_tokenize(text)

print("\nTokens:\n", tokens)

stop_words = set(stopwords.words('english'))

filtered_words = [word for word in tokens if word.lower() not in stop_words and word.isalpha()]

print("\nAfter Removing Stop Words:\n", filtered_words)

ps = PorterStemmer()

stemmed_words = [ps.stem(word) for word in filtered_words]

print("\nAfter Stemming:\n", stemmed_words)

tags = pos_tag(tokens)

print("\nPart-of-Speech Tags:\n", tags)

tag_counts = Counter(tag for word, tag in tags)

print("\nPOS Tag Counts:\n", tag_counts)

bigrams_list = list(bigrams(tokens))

trigrams_list = list(trigrams(tokens))

print("\nBigrams:\n", bigrams_list)

print("\nTrigrams:\n", trigrams_list)

fdist = FreqDist(bigrams_list)

print("\nBigram Frequencies:")

for bigram, freq in fdist.items():

```

```

print(bigram, ":", freq)
lidstone_prob = LidstoneProbDist(fdist, 0.1, bins=len(fdist))
print("\nSmoothed Probability Example:")
print("P('Tomatoes', 'are') =", lidstone_prob.prob(('Tomatoes', 'are'))))
grammar = "NP: {<DT>?<JJ>*<NN>|<NNS>}"
chunk_parser = RegexpParser(grammar)
chunked = chunk_parser.parse(tags)
print("\nChunked Sentence Tree:")
print(chunked)

```

OUTPUT

```

Original Text:
Tomatoes are essential in cooking. They grow well in warm climates.

Tokens:
['Tomatoes', 'are', 'essential', 'in', 'cooking', '.', 'They', 'grow', 'well', 'in', 'warm', 'climates', '.']

After Removing Stop Words:
['Tomatoes', 'essential', 'cooking', 'grow', 'well', 'warm', 'climates']

After Stemming:
['tomato', 'essenti', 'cook', 'grow', 'well', 'warm', 'climat']

Part-of-Speech Tags:
[('Tomatoes', 'NNS'), ('are', 'VBP'), ('essential', 'JJ'), ('in', 'IN'), ('cooking', 'VBG'), ('.', '.'), ('They', 'PRP'), ('grow', 'VBP'), ('well', 'RB'), ('in', 'IN'), ('warm', 'JJ'), ('climates', 'NNS'), ('.', '.')]

POS Tag Counts:
Counter({'NNS': 2, 'VBP': 2, 'JJ': 2, 'IN': 2, '.': 2, 'VBG': 1, 'PRP': 1, 'RB': 1})

Bigrams:
[('Tomatoes', 'are'), ('are', 'essential'), ('essential', 'in'), ('in', 'cooking'), ('cooking', '.'), ('.', 'They'), ('They', 'grow'), ('grow', 'well'), ('well', 'in'), ('in', 'warm'), ('warm', 'climates'), ('climates', '.')]

Trigrams:
[('Tomatoes', 'are', 'essential'), ('are', 'essential', 'in'), ('essential', 'in', 'cooking'), ('in', 'cooking', '.'), ('cooking', '.', 'They'), ('.', 'They', 'grow'), ('They', 'grow', 'well'), ('grow', 'well', 'in'), ('well', 'in', 'warm'), ('in', 'warm', 'climates'), ('warm', 'climates', '.')]

```

```

Bigram Frequencies:
('Tomatoes', 'are') : 1
('are', 'essential') : 1
('essential', 'in') : 1
('in', 'cooking') : 1
('cooking', '.') : 1
('.', 'They') : 1
('They', 'grow') : 1
('grow', 'well') : 1
('well', 'in') : 1
('in', 'warm') : 1
('warm', 'climates') : 1
('climates', '.') : 1

Smoothed Probability Example:
P('Tomatoes', 'are') = 0.08333333333333334

```

```

Chunked Sentence Tree:
(S

```

```

  (NP Tomatoes/NNS)
  are/VBP
  essential/JJ
  in/IN
  cooking/VBG
  ./
  They/PRP
  grow/VBP
  well/RB
  in/IN
  warm/JJ
  (NP climates/NNS)
  ./)

```

RESULT: Program executed successfully and output is verified.